DECODE LAB INTERNSHIP

Artificial Intelligence – Project 4

Text Recognition using Optical Character Recognition (OCR)

Intern Name	Shruti
Internship	Decode Lab Internship
Project	Project 4 – Image or Text Recognition (Basic)
Selected Path	Optical Character Recognition (OCR)
Primary Libraries	Python, OpenCV, pytesseract

1. Project Overview

This project implements a basic text-recognition workflow using Optical Character Recognition (OCR). The objective is to ingest an image, preprocess the visual data, extract machine-readable text, and display/save the result clearly. The project follows the Project 4 guidance supplied by DecodeLabs, which allows an OCR path using pytesseract.

2. Objective

The main objective is to build a functional recognition pipeline that can read text from a sample image. The workflow demonstrates the use of an available OCR library, image preprocessing, recognition, and clear output generation.

3. Technologies Used

• Python 3.x
• OpenCV (opencv-python) for image loading and preprocessing
• pytesseract as the Python wrapper for the Tesseract OCR engine
• Tesseract OCR installed separately on the computer

4. Methodology

The implementation follows these stages:
1. Load the input image.
2. Convert the image to grayscale.
3. Apply Gaussian blur to reduce small noise.
4. Apply thresholding to improve foreground/background separation.
5. Pass the processed image to Tesseract through pytesseract.
6. Display the recognized text and save it to an output file.

5. Pre-Processing

Raw images can contain shadows, uneven lighting, and visual noise. Grayscale conversion removes color information that is not required for basic OCR. Gaussian blur can reduce small artifacts, while thresholding produces a stronger contrast between text and background.

6. OCR Configuration

The implementation uses Tesseract's page segmentation mode (PSM) for a single uniform block of text. The DecodeLabs material explains that PSM selection depends on the layout of the input, so the setting can be changed when the image contains a different text arrangement.

7. Output

The program prints the extracted text in the terminal and saves the recognized text to output.txt. A processed image is also saved so the preprocessing result can be visually checked.

8. Validation

The supplied Project 4 material identifies four validation areas: library integration, preprocessing integrity, accuracy benchmarking, and visual confirmation. For an OCR implementation, the output should be checked against the original sample image for legibility and recognition quality. The source material specifies an 80% minimum confidence standard for the project; the exact achieved confidence must be measured on the chosen sample input rather than assumed.

9. Project Structure

main.py – main OCR program
sample_text.jpg – sample input image
output.txt – extracted text
processed_image.png – preprocessed image
README.md – project instructions
requirements.txt – Python dependencies

10. Conclusion

This project demonstrates a basic machine-perception workflow for converting visual text into machine-readable text. It applies image preprocessing and a pre-trained OCR engine instead of training a model from scratch. The project provides a practical implementation of the Image or Text Recognition (Basic) milestone described in the DecodeLabs Project 4 material.

Reference: DecodeLabs Artificial Intelligence Project 4 – Industrial Training Kit, Batch 2026.